

CityTech TTP Summer Bootcamp

{topic}

2026-06-08

Agenda

1. Review

2. Intro to React

Review

What are the most important things you remember from last week?

Why React Emerged

- Early web: **static pages**, then jQuery-era interactivity — hand-editing the DOM, step by step
- As **SPAs** grew, keeping the screen in sync with the data by hand became messy and bug-prone
- **Facebook open-sourced React in 2013** to manage complex, constantly-updating UIs (news feed, chat, notifications)
- Today it's one of the most widely used ways to build web interfaces

What Is React?

- A **JavaScript library** for building user interfaces
- **Components** — reusable, self-contained pieces of UI
- **Declarative** — you describe what the UI should look like; React updates the page for you
- **One-way data flow** — data passes down through **props**
- **JSX** — write HTML-like markup right inside JavaScript

The Big Idea: $UI = f(state)$

Instead of manually changing the DOM, you write your UI as a function of the current **state**.
Change the state → React re-renders just what changed.

```
function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>;  
}
```

This is the SPA idea in practice: only the component that changed re-renders.

Real-World React Frontends

Open-source React apps worth exploring:

- **Excalidraw** — virtual whiteboard, client-side React + TypeScript
<https://github.com/excalidraw/excalidraw>
- **tldraw** — infinite canvas and drawing
<https://github.com/tldraw/tldraw>
- **TodoMVC** — the classic to-do app, small and readable
<https://github.com/tastejs/todomvc>

What Is Boilerplate?

Boilerplate is the standard, repeated setup code and config every project needs *before* you write any features — folder structure, build config, linting, base dependencies.

- The same scaffolding shows up across countless projects
- Teams start from **starter templates / generators** instead of reinventing it
- Saves time and enforces consistency across a codebase
- Trade-off: it can include things you don't need — know what it sets up

Start a React + TypeScript Project

We use **Vite** (modern and fast) with the React + TypeScript template — and **Yarn**, not npm.

```
yarn create vite my-app --template react-ts
cd my-app
yarn          # install dependencies
yarn dev      # start the dev server
```

- `react-ts` gives you React + TypeScript boilerplate, ready to go
- Open the printed `localhost` URL to see it live

Vite has replaced the older `create-react-app` as the go-to scaffolder.

`src/` vs. `dist/`

- `src/` — your source code: the files you write (TypeScript, JSX, CSS)
- `dist/` — the build output: optimized, browser-ready files generated from `src/`
- You **edit** `src/`; you **never hand-edit** `dist/` — it's regenerated each build
- The browser ultimately runs what's in `dist/`

Compilation and Building

Compilation turns your source into what browsers can actually run: TypeScript → JavaScript, JSX → JS — then it's bundled and minified.

```
yarn build          # compile src/ → dist/ once  
yarn build --watch  # rebuild automatically on every save
```

- `yarn build` runs the project's build script (Vite) to produce `dist/`
- `--watch` keeps watching your files and rebuilds as you change them

What Is Yarn? (vs. npm)

Both are **package managers** — they install and manage the libraries your project depends on (everything in `node_modules`).

- **npm** ships with Node; **Yarn** is a popular alternative — the one we use
- Both read `package.json` and write a lockfile for reproducible installs (`yarn.lock` vs. `package-lock.json`)

```
npm install           → yarn
npm install <pkg>     → yarn add <pkg>
npm run <script>      → yarn <script>
npm uninstall <pkg>  → yarn remove <pkg>
```

Why SPAs? (Reminder)

Traditional sites reload the **whole page** from the server on every click. A **single-page app (SPA)** loads once, then updates only the parts that change.

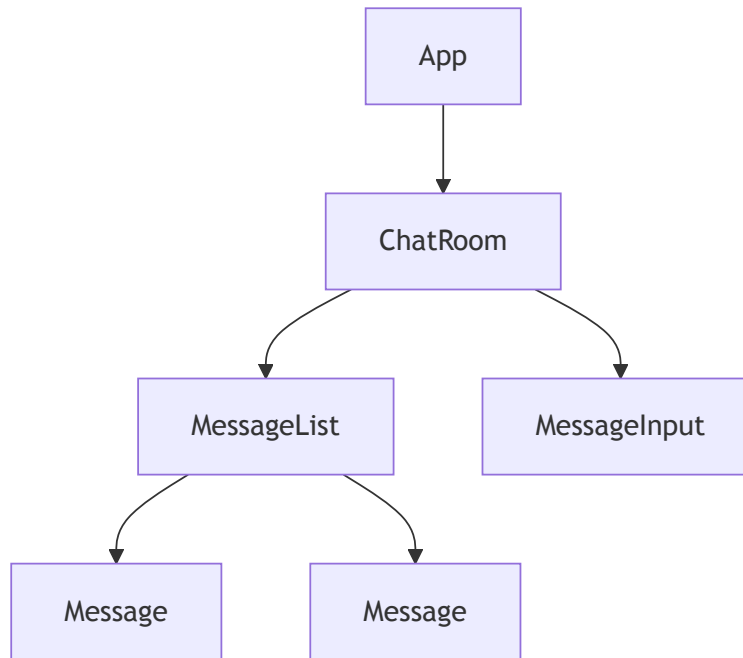
- **App-like feel** — no jarring full-page reloads
- **Less data** after the first load — fetch just the data, not whole pages
- **UI keeps its state** as you move around
- React is built for this: components re-render when their data changes

Components: The Mental Model

- A **component** is a function: **props in** → **UI (JSX) out**
- You build your interface as a **tree** of nested components
- They're **reusable building blocks** — think LEGO bricks
- A component can hold its own **state** and re-renders when that state changes

Composing Components

Break a screen into components, then nest them into a tree. **Props flow down** (parent → child); **events flow up** (child → parent via callbacks).



Code Together — Build a Fake Chatroom

Let's build a simple chat UI together — no real backend, messages just live in React **state**.

- **ChatRoom** — holds the list of messages in state
- **MessageList** — renders all the messages
- **Message** — displays a single message
- **MessageInput** — text box + send button that adds a message

"Fake" = messages are stored in state, not sent anywhere.